

SMART AGRICULTURE ASSISTANT

Backend Architecture & Technical Documentation

A comprehensive code-level review and flow documentation

Project Type: Node.js, Express, MongoDB (CommonJS & ESM Hybrid)

Language Support: English, Marathi (मराठी), Hindi (हिंदी)

Generated on: 15 August 2026

Created by: Antigravity AI Coding Assistant

1. Executive Summary & System Overview

The Smart Agriculture Assistant backend is designed as an interface helper for farmers, bridging the gap between field sensors/inputs and digital intelligence. The application acts as a central aggregator that processes critical soil NPK metrics, weather parameters, and crop types to generate localized, translation-friendly agricultural insights.

The server is built with Node.js and Express.js, integrating MongoDB as the primary persistence layer through Mongoose ODM. To cater to Indian farmers, particularly in Maharashtra, the application embeds a translation dictionary enabling responses in Marathi (मराठी) and Hindi (हिंदी). Furthermore, it implements fallbacks so the server remains functional with mock datasets even if MongoDB or external AI endpoints are unavailable.

Core System Components:

- **API Endpoints (Routes):** Handles client HTTP requests, validates input parameters, invokes services, and sends JSON data back.
 - **Agriculture Services:** Encapsulates mathematical formulas and evaluation thresholds for NPK ranges and pH ratings.
 - **Translation Module:** Translates crops, pests, and messages dynamically before sending them to the client.
 - **Database Schemas (Models):** Defines rules for persistent documents like expert profiles, consultations, shop locators, and query logs.
 - **External Integrations:** Connects to Google Gemini and OpenAI for semantic chatbot conversations, and Plant.id for image disease analysis.
-

2. Technology Stack & Key Dependencies

The backend runs on an asynchronous Node.js engine, utilizing ES Modules (import/export) for its main runtime execution and CommonJS formatting for its database schemas. Below are the key dependencies defined in package.json:

Package	Version	Role in the Project
express	^4.18.2	HTTP server router, middleware, endpoints orchestration
mongoose	^9.1.2	Object Document Mapper (ODM) for MongoDB database schemas
multer	^1.4.5-lts.1	Middleware for handling multipart/form-data (image uploads)
axios	^1.6.2	Promise-based HTTP client for external API calls (Gemini/OpenAI/...
cors	^2.8.5	Configures cross-origin requests from web frontends & mobile
dotenv	^16.3.1	Loads configuration variables from .env or config.env files
body-parser	^1.20.2	Parses incoming request bodies (URL-encoded and JSON objects)
nodemon (dev)	^3.0.2	Monitors file modifications and automatically restarts server
concurrently (dev)	^8.2.2	Allows executing backend and client frontend simultaneously

Configuration Management (.env / config.env):

Variables like PORT (default 5001), MONGODB_URI (IP database mapping), and API keys (GEMINI_API_KEY, OPENAI_API_KEY, CROP_DIAGNOSIS_API_KEY) are extracted from configuration environment files at launch. If they are absent, fallback paths are selected automatically without server crashes.

3. Database Schemas & Mongoose Models

The models define structure, validation rules, and indexes for data collections stored in MongoDB. There are 7 distinct schemas configured under `/models/`:

Expert.js

Fields: `expert_id`, `name`, `phone`, `email`, `specialization` (Array), `category`, `experience`, `rating`, `availability` (`status`, `working_hours`), `location` (`city`, `state`, `full`), `verified`

Stores profiles of crop pathologists, entomologists, and soil experts available for consultation.

ExpertRequest.js

Fields: `request_id`, `farmer_name`, `farmer_phone`, `expert_id`, `issue_description`, `image_url`, `status` (`pending`/`scheduled`/`completed`/`cancelled`), `scheduled_time`, `created_at`

Registers bookings made by farmers to consult experts regarding crop issues.

Store.js

Fields: `store_id`, `name`, `type` (`seeds`/`fertilizers`/`organic`), `location` (GeoJSON Point), `address`, `contact`, `products` (Array), `rating`, `hours`, `verified`

Maintains records of agricultural input stores. Coordinates are stored in GeoJSON format to enable geospatial (`$nearSphere`) search.

Query.js

Fields: `type` (`crop`/`soil`/`pest`/`etc.`), `language`, `requestData` (Mixed), `responseData` (Mixed), `timestamp`, `userLocation`

Logs transaction requests and responses to monitor API performance and build query analytics.

SoilAnalysis.js

Fields: `nitrogen`, `phosphorus`, `potassium`, `ph`, `soil_type`, `health_status`, `recommendations` (Array), `deficiencies` (Array), `language`

Stores history of soil test parameters and returned health diagnoses.

SystemConfig.js

Fields: `key`, `value`, `description`, `last_updated`

Contains dynamic application settings, enabling quick configuration adjustments without code redeployments.

PestDetection.js

Fields: `crop`, `pest_description`, `identified_pest`, `control_methods`, `organic_solutions`, `chemical_solutions`, `language`

Maintains records of pest descriptions and recommendations generated.

4. Core Business Logic (agricultureService.js)

The `/services/agricultureService.js` module serves as the primary engine for agricultural assessments. It runs raw logic and maps inputs to recommendations without database overhead. The key functions include:

1. Crop Recommendation (recommendCrops)

Accepts Nitrogen (n), Phosphorus (p), Potassium (k), pH (ph), Temperature (temp), Rainfall (rain), and language. It loops through a local crop database (defining range values for 10 major crops: rice, wheat, cotton, sugarcane, soybean, maize, groundnut, onion, potato, tomato). For each crop, it scores matching attributes out of 6. If a crop scores $\geq 4/6$, it is classified as suitable. The top 3 are returned, translated into Marathi/Hindi if requested.

2. Soil Health Analysis (analyzeSoil)

Accepts NPK, pH, and soil type. It diagnoses nutrient deficiencies (e.g., Nitrogen < 50 , Phosphorus < 30 , Potassium < 30) and flags pH values outside the optimal 6.0–7.5 range (suggesting lime for acidic soil and gypsum for alkaline soil). An overall health score is calculated: a blend of nutrient abundance and pH stability, classifying soil as Excellent, Good, or Needs Improvement.

3. Fertilizer Suggestion (getFertilizerRecommendation)

Recommends custom NPK blends based on crop growth stages. Under the seedling stage, starter fertilizer NPK 10:26:26 is proposed. Under the vegetative stage, high-nitrogen Urea is suggested. For flowering, potash-rich NPK 12:32:16 is recommended, complete with application safety warnings (avoid hot afternoons, verify water levels). All recommendations support English, Marathi, and Hindi.

Marathi Translation Integration (utils/translations.js):

A static translation dictionary maps crops (e.g., "Wheat" to "गहू"), soil types (e.g., "Sandy" to "वाळूमाती"), and diseases/pests (e.g., "Whitefly" to "पांढरी माशी"). The service files import `translateCropName` and `translateTerm` to deliver localized UI messages.

5. API Endpoints & Route Handlers

The `/routes/` directory contains 13 routing files. These are mounted on `/api` paths in `server.js`:

`/api/crop/recommend (POST)`

Validates NPK input and returns suitable crops matching local NPK constraints. Saves recommendations to database.

`/api/fertilizer/recommend (POST)`

Calculates specific fertilization schedules and ratios based on crop type and current plant lifecycle phase.

`/api/soil/analyze (POST)`

Accepts soil metrics, performs health checks, lists nutrient deficits, and recommends fertilizers or amendments.

`/api/pest/detect (POST)`

Matches pest descriptions to suggest organic treatments (e.g., Neem oil) or chemical alternatives.

`/api/market/prices (GET)`

Returns price indices for various crops per quintal based on mandi price benchmarks.

`/api/weather/:location (GET)`

Generates mock weather data and issues warnings (e.g. moderate rain predicted, proceed with sowing).

`/api/stores/nearby (POST)`

Accepts location coordinates and searches for nearby agricultural supply shops using MongoDB geospatial indices.

`/api/expert/list (GET) & book-consultation (POST)`

Handles scheduling, availability checks, and booking expert agricultural support.

`/api/expert-request/request (POST)`

Farmer-focused requests submitting plant health details. Supports status monitoring and cancellations.

`/api/system/config (GET/PUT)`

Endpoint allowing administrators to modify system variables dynamically.

6. AI Chatbots & Image-Based Disease APIs

The AI components connect the server to LLM engines and crop-identification systems to dynamically resolve complex, unstructured questions.

AI Chatbot Routes (/api/chatbot/chat)

The chatbot supports multi-lingual prompts. It utilizes Google Gemini (gemini-1.5-flash) as the primary engine. If the Gemini API key is missing or fails due to rate limits, it falls back to OpenAI (gpt-3.5-turbo). If both integrations are offline, it defaults to a preconfigured friendly assistant message. The query and generated text are logged to the ChatHistory database schema for audit records.

Plant Disease Detection (/detect)

This route processes physical crop images uploaded by farmers. It uses Multer middleware to stream and write files to the /uploads/ folder. The route operates under a tier structure:

- **Tier 1: Plant.id Health API:**

If CROP_DIAGNOSIS_API_KEY is defined, the server converts the image to base64 and invokes the Plant.id health assessment API to identify the disease, returning the confidence index and treatments.

- **Tier 2: Hash-Based Local Fallback:**

If the API is disabled or fails, the script uses the file size and a SHA-256 hash of the binary file to select a consistent mock disease (e.g. Tomato Early Blight, Rice Blast, Potato Late Blight, or Healthy Plant). This ensures that the same uploaded image always produces the exact same diagnosis during testing, providing stable local mock results.

7. server.js Line-by-Line Breakdown

The server.js file acts as the main program entry. Let us inspect its core segments line-by-line:

[Lines 1 – 4] – ESM Imports

Imports core packages: express for Routing, cors for web permissions, mongoose for database mapping, and dotenv for environment files.

[Line 7] – Loading Config Environment

Calls `dotenv.config({ path: './config.env' })` to pull environment variables like PORT, API_KEYS, and MONGODB_URI into Node process.env.

[Lines 10 – 22] – Mounting API Route Modules

Imports 13 route files (e.g. cropRoutes.js, weatherRoutes.js). The file extensions .js are required as the package uses ESM modules.

[Lines 24 – 55] – CORS Configuration & Express Init

Initializes the express app. CORS middleware is configured to allow requests from local network addresses (port 3000, 5173, 8080) for hybrid mobile/web environments.

[Lines 57 – 64] – Middleware Registration

Configures logging middleware (prints incoming request methods and paths to the console) and mounts `express.static('.')` to serve local web interface files.

[Lines 67 – 92] – MongoDB Connection with Auto-Fallback

Resolves MONGODB_URI. Connects via `mongoose.connect` with a 30-second `serverSelectionTimeoutMS`. If database is offline, it catches the error and switches to "Mock Data Mode", letting endpoints function without crashes.

8. server.js Breakdown & Seeding Scripts

server.js Route Mapping & Server Execution:

[Lines 103 – 112] – GET /health Check

Exposes server health. Returns a JSON object containing database connection status, local system timestamp, and server uptime.

[Lines 114 – 159] – GET / Endpoint Directory

Acts as root directory, displaying server status, welcome banner, and a comprehensive dictionary mapping of all 30+ endpoints.

[Lines 162 – 174] – Mounting API Sub-Routers

Attaches sub-routers under namespace prefixes (e.g. `app.use("/api/crop", cropRoutes)`).

[Lines 176 – 193] – Error Handlers

Registers a global try-catch handler returning 500 status on unhandled throws, followed by a 404 handler for unknown URL paths.

[Lines 196 – 223] – Server Execution and EADDRINUSE Handler

Starts express on PORT. Registers an error event listener: if port 5001 is already bound, it catches EADDRINUSE, suggestions solutions (`kill-port.bat`), and exits gracefully.

[Lines 226 – 247] – Graceful Shutdown Hooks

Listens to SIGTERM and SIGINT events (such as Ctrl+C). On trigger, it stops the Express server, closes the MongoDB connection, and exits to ensure zero resource leaks.

Database Seeding: init-db-native.js

A Native MongoClient seeding script designed for stability. It connects to the Demo database on the configured IP, verifies document counts in the experts and stores collections, and inserts sample expert profiles and GeoJSON-ready coordinates for agricultural shops. Running `node init-db-native.js` sets up the initial environment.

9. Running Locally & Helper Batch Scripts

The project contains several helper utility files designed to facilitate execution and debugging on Windows platforms.

- **start-server.bat:**

Double-clicking runs the dev script (npm run dev) utilizing nodemon to start on Port 5001.

- **kill-port.bat:**

Queries netstat to locate the process ID occupying port 5001 and terminates it using taskkill. Useful when the node process gets stuck in the background.

- **test-connection.bat:**

Tests network connection directly, verifying if the backend ports are responding.

- **test-fertilizer.bat:**

Invokes powershell scripts to run API test requests against the fertilizer endpoints.

- **test-fertilizer-api.html:**

A local HTML dashboard containing form elements to manually input soil conditions and visually test API recommendations.

Troubleshooting Commands:

npm run init-db — Seeds local database with standard experts and stores

npm run dev — Launches backend with auto-reload activated

node test-mongo.js — Validates raw connection stability to local MongoDB server

node reset-database.js — Clears all collections to enable a clean seeding environment

CONFIDENTIAL – STUDY REFERENCE

CONFIDENTIAL – STUDY REFERENCE

CONFIDENTIAL – STUDY REFERENCE

CONFIDENTIAL – STUDY REFERENCE

CONFIDENTIAL – STUDY REFERENCE

CONFIDENTIAL – STUDY REFERENCE

CONFIDENTIAL – STUDY REFERENCE

CONFIDENTIAL – STUDY REFERENCE

CONFIDENTIAL – STUDY REFERENCE

